

Triana — Bot de cobranza automatizada por WhatsApp

Resumen del proyecto: visión, flujo de funcionamiento y estado actual

Documento generado el 09/07/2026

Triana es un bot que automatiza el cobro de cuotas de ventas a plazo (por ejemplo, muebles pagados en cuotas por Pix). Registra las ventas, recuerda a cada cliente cuándo le toca pagar, lee automáticamente la foto del comprobante que manda por WhatsApp, confirma el pago sin intervención manual, y todos los días te envía un resumen con lo cobrado y lo pendiente.

1. Flujo completo de funcionamiento

- 1 Registro de la venta.** Al vender un producto se carga: nombre y teléfono del cliente, producto, monto total, número de cuotas y valor de cada cuota.
- 2 Recordatorio automático.** Cada día el sistema revisa quién debe pagar hoy o está atrasado y le manda un mensaje de WhatsApp.

"Hola Juan, hoy vence tu cuota #3 de R\$100 😊
Puedes pagar por Pix y enviar el comprobante aquí"

- 3 El cliente envía el comprobante** como foto por WhatsApp.
- 4 Lectura automática.** El sistema usa OCR (Tesseract) para leer el texto de la imagen y una IA (OpenAI) para interpretar el monto pagado y el nombre del pagador, comparando contra la cuota pendiente en la base de datos.
- 5 Confirmación automática.** Si el monto coincide, se registra el pago y se avisa al cliente.

"Pago recibido ✅
R\$100 registrado
Ahora estás en la cuota 4/10"

- 6 Manejo de dudas.** Si el monto no coincide o hay un error de lectura, el bot le pide al cliente que confirme el valor manualmente en vez de fallar en silencio.

- 7 Informe diario.** Todos los días a la noche, el sistema te manda a vos (el administrador) un resumen por WhatsApp.

 Resumen del día (28/04)

✓ Pagos recibidos:
Juan → R\$150 (cuota 3)
María → R\$200 (cuota 5)

⚠ Pendientes:
Carlos → 2 días de atraso
Ana → vence hoy

💰 Total recibido: R\$350

2. Decisiones de plataforma tomadas

COMPONENTE	HERRAMIENTA ELEGIDA	POR QUÉ
Backend	Node.js + Express	Ya estaba en marcha; liviano para correr los cron jobs y la API.
WhatsApp	whatsapp-web.js no oficial	Gratis, se conecta escaneando un QR (como WhatsApp Web). Se eligió para validar la idea rápido con pocos clientes. Queda pendiente evaluar migrar a la API oficial (Twilio / 360dialog) si el volumen crece, porque hay riesgo real de que Meta banee el número.
Base de datos	Supabase (Postgres)	Gestionada en la nube, gratis para empezar, con Storage integrado para guardar las fotos de los comprobantes (antes se guardaban como texto base64 en la base, lo cual no escala).
OCR	Tesseract.js	Gratis y corre local. Se deja la puerta abierta a Google Cloud Vision más adelante si la precisión no alcanza.
IA	OpenAI (GPT-3.5)	Interpreta la intención de los mensajes de texto y extrae monto/nombre del comprobante leído por OCR.
Panel de control	React (frontend)	Dashboard, clientes, ventas y pagos — en construcción.

3. Estado actual del backend

Hecho ✓

- ✓ Esquema de base de datos (clientes, cuotas, pagos) listo para Supabase

Pendiente ○

- Crear el proyecto real en Supabase y cargar las credenciales

- ✓ Bucket de Storage para comprobantes de pago
- ✓ Capa de datos migrada de MySQL a Supabase
- ✓ Registro de ventas y cuotas vía API
- ✓ Envío de recordatorios de cobro por cron diario
- ✓ Lectura de comprobantes: OCR + IA con respaldo automático por regex
- ✓ Confirmación de pago y actualización de cuota
- ✓ Generación del informe diario
- ✓ Normalización de números de teléfono (WhatsApp vs base de datos)
- ✓ El bot ignora grupos y solo responde en chats individuales
- ✓ Punto de entrada del servidor (arranca API + WhatsApp + cron jobs juntos)
- Probar el flujo completo con un cliente y un comprobante real
- Terminar el panel (Dashboard, Clientes, Pagos, Ventas) en React
- Definir si más adelante se pasa a la API oficial de WhatsApp Business
- Elegir dónde desplegar el backend para que corra 24/7 (Railway, Render, VPS)
- Decidir cómo se auto-registran clientes nuevos que escriben sin haber sido cargados antes

4. Próximos pasos sugeridos

1. Crear la cuenta y el proyecto en Supabase, correr el script de esquema, y completar el archivo `.env`.
2. Levantar el backend local (`npm install + npm run dev`) y escanear el QR de WhatsApp.
3. Registrar un cliente de prueba y simular el ciclo completo: recordatorio → envío de comprobante → confirmación → informe diario.
4. Avanzar con el panel de control en React para no depender solo de WhatsApp para cargar ventas.
5. Con datos reales de uso, decidir si conviene migrar a la API oficial de WhatsApp Business.